

EXAMEN DE FIN D'ÉTUDES SECONDAIRES – Sessions 2024**QUESTIONNAIRE**

Date :	17.09.24	Horaire :	08:15 - 11:15	Durée :	180 minutes	
Discipline :	SCIPR	Type :	écrit	Section(s) :	CI	
					Numéro du candidat :	

Modelling part

10 points

- The modeling part must be written on paper and must be handed in before the implementation part can start.
- The modeling part is due after 35 minutes at the latest, but you are free to hand it in earlier.

A manager from a bank comes to you and wants to digitalize his banking transactions. He wants an application that allows his employees to create different user accounts and make transactions between user accounts. Since the bank has many customers, the manager wants the application to work as quickly as possible.

Description of the Application

The bank contains a data structure to store accounts and another data structure to store transfers. An account consists of a unique account number, the username of its owner, the currency and a balance. The same username can be used in multiple accounts. Since the bank must manage lots of accounts it is essential to be able to retrieve all accounts with the same username very quickly. It must also be possible to compare different accounts with each other.

A transfer contains a sender, a receiver account, the amount of money transferred from one account to the other and the currency of the money transferred. Before a transfer is performed it needs to be validated. A transfer is valid if the sender and receiver are different accounts, both accounts have the same currency and the sender has enough funds. The order in which the transfers are performed and added to the data structure is important to properly track when which transfer was performed.

It is also required that the main currency of the accounts with the same username is determined. The main currency is the currency with the highest total balance.

The screenshot shows a 'Bank' application window with three main sections:

- User selection:** A text field for 'Username' containing 'Max' and a 'Select' button.
- Account Management:**
 - Fields for 'Currency' (€) and 'Balance' (800).
 - A 'Create Account' button.
 - A 'Main Currency' field set to '€'.
 - A text area titled 'All Accounts of user: Max' showing:
 - Account: 643 contains: 745.0 \$
 - Account: 744 contains: 208.0 €
 - Account: 920 contains: 542.0 €
- Transfer Window:**
 - Fields for 'Sender number' (744), 'Receiver number' (920), and 'Amount to transfer' (42).
 - A 'Transfer' button.
 - A text area titled 'Transfer History:' showing:
 - 744 -> 920 : 42.0 €
 - 643 -> 305 : 5.0 \$
 - 643 -> 305 : 50.0 \$

The bank requires that the application has the following functions:

1. The application displays all the accounts that contain the specified username.
2. The accounts are displayed as is shown in the screenshot.
3. The application must be able to verify and perform a transfer from one account to another.
4. The application must be able to create a history of transfers.
5. The transfers are displayed as shown in the screenshot.
6. The application must determine which is the main currency of a given user.

Work to be done:

Adding comments onto your diagram is recommended in order to explain your choice!

Produce a class model of the application model on paper, following the usual UML conventions.

1. First add the required classes and their attributes.
2. Add the method(s) needed to sort the accounts using Java's **Collections.sort()** method.
3. Add the method(s) required to display all the accounts of a single user in text form in a list, as shown in the screenshot.
4. Add the method(s) required to display all the transfers in text form in a list, as shown in the screenshot.
5. Add the method(s) required to determine the main currency of the accounts containing the same username.
6. Add the method(s) required to validate a transfer.

In your working directory (to be defined by each school), you'll find a folder named EXAMEN_CI. Rename this folder by replacing the current name with your exam code (example notation: LXY_CI2_05). All your files should be saved inside this folder, which will be called your folder in the following.

Implementation part

50 points

In the following section, you are going to program a simplified simulation of a banking system that manages several bank accounts and transfers. In your folder, you will find the **Bank** project, which you must complete.

The "Account" class

(1.5 points)

The **Account** is partially provided. This class represents a user's bank account. The constructor is provided as well as the necessary getters and setters. The constructor initializes the value of the **number**, **username**, **balance** and **currency** attributes. The class has the following attributes:

- **number** represents the identifying number of the account
- **username** corresponds to the name of the owner of the account
- **balance** represents the current balance of the account
- **currency** indicates which currency is stored in this account

The class has the following methods:

- The **receiveMoney(double)** method is provided and adds the specified amount to the balance.
- The **withdrawMoney(double)** method is provided and reduces the balance by the specified amount.
- It implements the **Comparable<Account>**. 0.5 points
- The **compareTo(Account)** method compares two accounts using their account numbers. The accounts must be sorted in ascending order. 0.5 points
- The **toString()** method returns a text representation of the account in the following format: 0.5 points

Account: <accountNumber> contains: <balance> <currency>

The "Transfer" class**(3 points)**

Complete the **Transfer** class which is used to perform a transfer from one account to another and store the relevant information. The constructor is provided and initializes its attributes:

- **sender** represents the sending account
- **receiver** represents the receiving account
- **amount** represents the amount of currency to be transferred
- **currency** represents the currency that is traded

Implement the following methods:

- The **perform()** method checks if a transfer is valid and performs it, by withdrawing money from the **sender** account and sending it to the **receiver** account. A transfer is valid if both accounts use the same **currency**, it doesn't send money to itself and if the **sender** account has enough funds. If a transfer can't be performed an exception of the type **InvalidTransferException** (class provided) is thrown with a significant message. **2 points**
- The **reverseTransfer()** transfers the sent amount from the **receiver** account back to the **sender** account without any further checks. **0.5 points**
- The **toString()** method returns a text representation of the transfer in the following format: **0.5 points**

<senderNumber> -> <receiverNumber> : <amount> <currency>

Example:

123 -> 355 : 500 €

The "Node" class**(1 point)**

The **Node** class represents a node in a linked list used to store transfers. Add the **Node** class according to the UML diagram. The **constructor** initializes the value of the **data** attribute. It has the following attributes: **1 point**

- **next** represents the next node
- **data** represents the transfer

The "TransferStack" class**(6.5 points)**

The **TransferStack** class is a stack containing the history of transfers. Add the **TransferStack** class according to the UML diagram. It has the following attributes:

0.5 points

- **top** represents the top element of the stack
- **size** represents the number of elements in the stack

The class has the following methods:

- The **push(Transfer)** method adds the given transfer on top of the stack. 1.5 points
- The **reverseAndPop()** method checks if the stack is empty. In that case an exception of the type **InvalidElementException** (class provided) is thrown with a significant message. If no exception is thrown the transfer on top of the stack is reversed, removed and returned. 2.5 points
- The **toArray()** method returns an Object array containing the transfers of the stack in the same order. 2 points

The "Bank" class**(33 points)**

The **Bank** class represents a bank management system. It contains all the accounts and enables money transfers from one account to another. Add the **Bank** class according to the UML diagram.

1 point

- **hmAccounts** is a map that maps the username to a list of its accounts
- **transferStack** is a stack that contains all transfers

The class has the following methods:

- **loadFromFile(String)** method loads already existing bank accounts and the transactions history from a CSV gives as parameter and stores them in their respective data structure. Each line in the CSV is either an account or a transfer. An account starts with the letter "A" and a transfer with the letter "T". 6.5 points

Structure account:

A,<number>,<username>,<balance>,<currency>

Structure transfer:

T,<sender>,<receiver>,<amount>,<currency>

If the first letter is neither an "A" nor a "T" then an exception of the type **InvalidElementException** (class provided) with a significant message is thrown.

- The **binarySearch(ArrayList<Account>, int)** method searches for the account with the specified **number** in the specified list using the *binary search algorithm* and then returns it. If the **Account** is not found, then **null** is returned. 5 points
- The **getAccountByNumber(int)** method returns the **Account** associated to the given **accountNumber** by searching all accounts in the map. 2.5 points
- The **checkAccountNumberAvailable(int)** method indicates whether an account could be found with the specified account number. 0.5 points
- **createNewAccountNumber()** is a method that returns a unique account number, between the values 100 and 999 included, that is still available. 2 points

- The **createAccount(String, String, double)** method creates a new account. First an available account number is determined. Then an **Account** is created with the determined account number and the specified username, balance and currency. Afterwards the account is added to the list of accounts of the specified username. Finally, the list of accounts is sorted. 1.5 points
- The **getUserAccounts(String)** method returns an object array containing all the accounts of the specified **username**. If no user exists with the specified username, then a new entry is created in the map. 1.5 points
- The **transfer(int, int, double)** method is responsible for making a transfer and adding it to the stack. If at least one of the provided account numbers is not valid, meaning there exists no account with that number, an exception of the type **InvalidElementException** (class provided) with a significant message is thrown. 3.5 points
- **reverseTransfers(int)** is used to reverse the specified number of transfers. 1.5 points
- The **transfersToArray()** method returns an object array containing all the transfers. 0.5 points
- **getPreferredCurrency(String)** is a method that is used to determine and return the currency with the highest total balance for the specified **username**. In the case that there are no accounts associated with the username, an empty String is returned. 7 points

The "MainFrame" class**(5 points)**

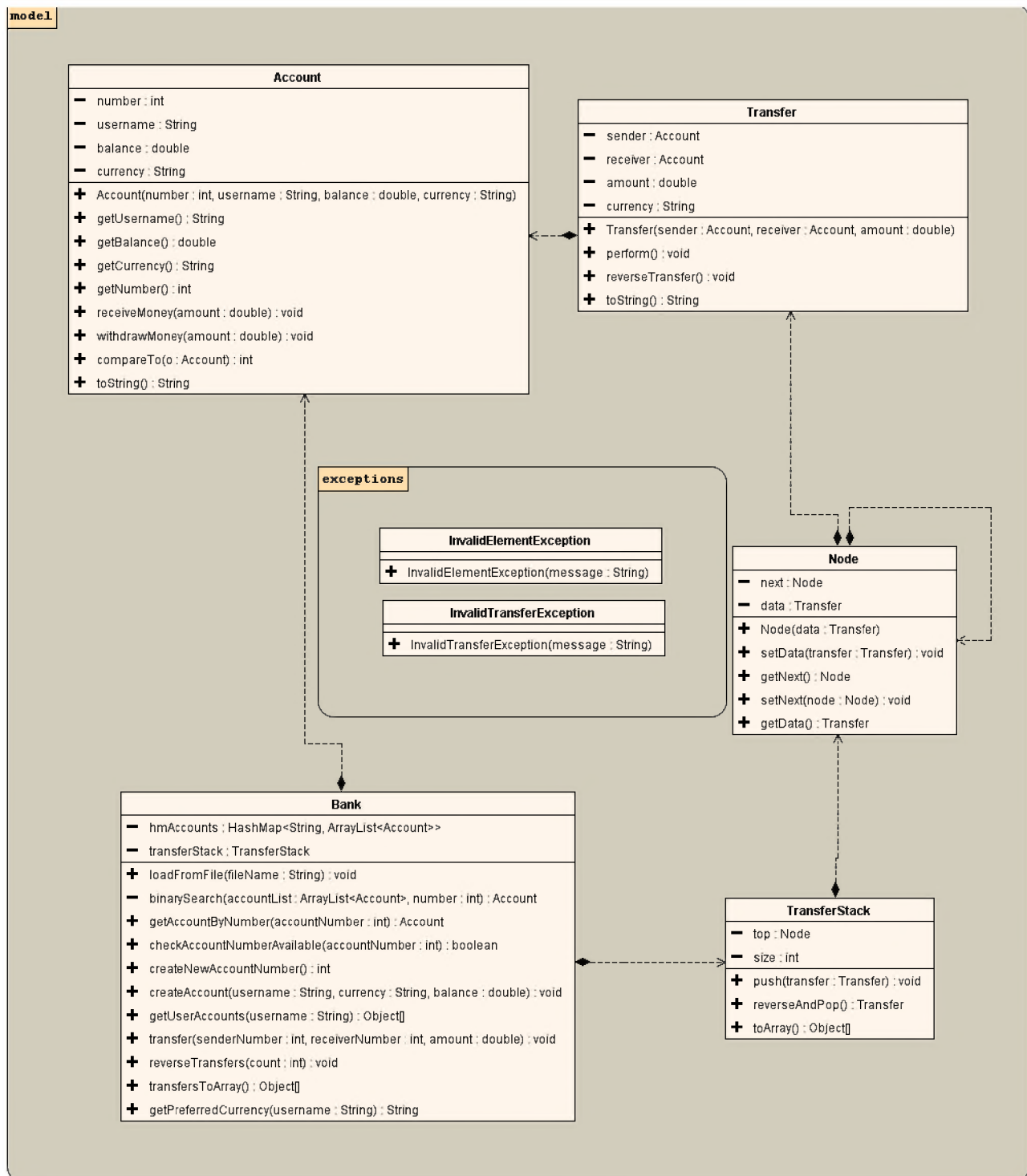
The **MainFrame** class is partially provided. Complete the class with the following instructions:

All exceptions thrown must be displayed in a window using the following code:

```
JOptionPane.showMessageDialog(null, "message");
```

This is provided in the **MainFrame** class.

- The **bank** attribute represents the application model and is initialized. **0.5 points**
- The constructor loads the existing accounts and transfer history from the CSV file "**bank.csv**". If an exception occurred during the reading of the file, then the message: "The provided file is corrupt or doesn't exist!" must be displayed! **1.5 points**
- The **updateView()** method is provided and updates the **accountsList**, **transferList**, **usernameLabel** and the **preferredCurrencyLabel** with data from the **bank** attribute.
You need to uncomment the code!
- The "**Show Account**" button is given and reads the input from the **usernameTextField** and updates the interface.
- The "**Create Account**" button is given and reads the currency and the balance from their corresponding text fields and creates a new account. Then the view is updated.
You need to uncomment the code!
- The "**Transfer**" button is partially provided and must be completed! This button reads the account number of the sender, of the receiver and the amount to be transferred from their respective text fields and then transfers the money from one account to the other using the **bank** attribute. **1.5 points**
- The "**Revert transfers**" button reverts the latest transfers up to the selected index in the **transferList**. **1.5 points**

UML diagram of the model to be implemented.

MainFrame
– bank : Bank – accountsList : javax.swing.JList – balanceTextField : javax.swing.JTextField – createAccountButton : javax.swing.JButton – currencyTextField : javax.swing.JTextField – jLabel1 : javax.swing.JLabel – jLabel2 : javax.swing.JLabel – jLabel3 : javax.swing.JLabel – jLabel4 : javax.swing.JLabel – jLabel5 : javax.swing.JLabel – jLabel6 : javax.swing.JLabel – jLabel7 : javax.swing.JLabel – jLabel8 : javax.swing.JLabel – jLabel9 : javax.swing.JLabel – jPanel1 : javax.swing.JPanel – jPanel2 : javax.swing.JPanel – jPanel3 : javax.swing.JPanel – jScrollPane1 : javax.swing.JScrollPane – jScrollPane2 : javax.swing.JScrollPane – preferredCurrencyLabel : javax.swing.JLabel – receiverNumberTextField : javax.swing.JTextField – revertTransfersButton : javax.swing.JButton – senderNumberTextField : javax.swing.JTextField – showAccountsButton : javax.swing.JButton – transferAmountTextField : javax.swing.JTextField – transferButton : javax.swing.JButton – transferList : javax.swing.JList – usernameLabel : javax.swing.JLabel – usernameTextField : javax.swing.JTextField
+ MainFrame() + updateView() : void – initComponents() : void – showAccountsButtonActionPerformed(evt : java.awt.event.ActionEvent) : void – createAccountButtonActionPerformed(evt : java.awt.event.ActionEvent) : void – transferButtonActionPerformed(evt : java.awt.event.ActionEvent) : void – revertTransfersButtonActionPerformed(evt : java.awt.event.ActionEvent) : void + <u>main(args[] : String) : void</u>